

Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Công nghệ Thông tin



Môn học: Xử lý Ngôn ngữ Tự nhiên
CS221.Q21.KHTN

Large Language Models

Nhóm 7

Nguyễn Thiên Bảo - MSSV: 23520127
Cao Lê Công Thành - MSSV: 23521437
Đặng Quang Vinh - MSSV: 23521786

Ngày 31, tháng 03, năm 2026

- 1 Three architectures for language models
- 2 Conditional Generation of Text: The Intuition
- 3 Prompting
- 4 Generation and Sampling

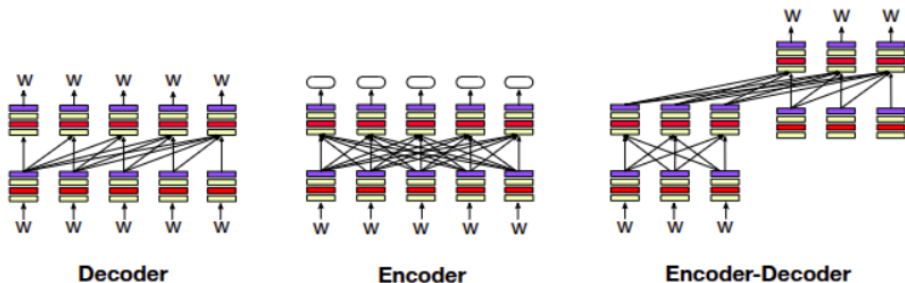
Sự khao khát giao tiếp & Khái niệm LLM

- **Khát vọng ngàn đời:** Con người luôn khao khát trò chuyện với các thực thể vô tri, thể hiện qua câu chuyện về bức tượng Pygmalion, Frankenstein, hay truyền thuyết về tượng Moses của Michelangelo.
- **Từ ELIZA đến LLM:**
 - Hệ thống ELIZA (mô phỏng nhà tâm lý học cách đây 60 năm) là minh chứng sơ khai cho việc giao tiếp với máy tính.
 - Ngày nay, LLM ra đời đóng vai trò là một tác tử tính toán (computational agent) chuyên biệt để tương tác hội thoại.

LLM hoạt động như thế nào? (Pretraining & Sinh văn bản)

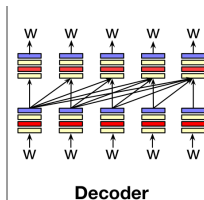
- **Cách học (Pretraining):** LLM học kiến thức về ngôn ngữ và thế giới đơn giản bằng cách liên tục được dạy để dự đoán từ tiếp theo trong các khối dữ liệu văn bản khổng lồ.
- **Giả thuyết phân bố:** Ý nghĩa của một từ được máy tính tiếp thu dựa trên sự liên kết phức tạp với các từ thường xuất hiện cùng với nó.
- **Sinh văn bản Tự hồi quy (Autoregressive Generation):**
 - Mô hình có khả năng sinh văn bản mới bằng cách "lấy mẫu" (sampling) từ phân bố xác suất của từ tiếp theo.
 - Quá trình tạo từ diễn ra từ trái sang phải.
 - Token vừa sinh ra lập tức được cộng gộp vào ngữ cảnh ban đầu để làm tiền đề dự đoán token tiếp theo tạo thành vòng lặp.

1. Three architectures for language models



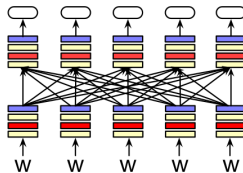
- Mô hình tự hồi quy (từ trái sang phải) chỉ là một trong ba kiến trúc mô hình ngôn ngữ phổ biến.
- Ba kiến trúc này bao gồm: **Bộ giải mã (Decoder)**, **Bộ mã hóa (Encoder)**, và **Bộ mã hóa - giải mã (Encoder-Decoder)**.

Kiến trúc Decoder (Bộ giải mã)



- **Cách hoạt động:** Nhận đầu vào là chuỗi token và lặp lại việc tạo ra từng token đầu ra ở mỗi bước.
- **Luồng thông tin:** Đi từ trái sang phải, nghĩa là mô hình chỉ dự đoán từ tiếp theo dựa trên các từ trong quá khứ.
- **Đặc điểm:** Đây là các mô hình tạo sinh (generative models), chuyên tạo ra các token hoàn toàn mới.
- **Ví dụ tiêu biểu:** Dùng để tạo ra các Mô hình Ngôn ngữ Lớn (LLM) như GPT, Claude, Llama, và Mistral.

Kiến trúc Encoder (Bộ mã hóa)



- **Cách hoạt động:** Nhận chuỗi token đầu vào và xuất ra một vector đại diện (encoding) cho mỗi token đó.
- **Cơ chế học:** Thường là mô hình ngôn ngữ bị che khuất (masked language models), đoán từ bị che bằng cách nhìn vào các từ xung quanh ở cả hai phía.
- **Đặc điểm:** Không phải mô hình tạo sinh văn bản; thường được tinh chỉnh (finetuning) để tạo thành các bộ phân loại như phân tích cảm xúc hay chủ đề.
- **Ví dụ tiêu biểu:** Các mô hình thuộc họ BERT như BERT, RoBERTa.

Kiến trúc Encoder-Decoder

- **Cách hoạt động:** Nhận một chuỗi token đầu vào và tạo ra một chuỗi token đầu ra.
- **Đặc điểm:** Dùng để ánh xạ giữa các loại token khác biệt nhau, với mối liên kết lỏng lẻo hơn giữa đầu vào và đầu ra.
- **Tính linh hoạt:** Chuỗi đầu ra có thể thuộc một tập từ vựng khác hoặc có độ dài ngắn khác biệt so với chuỗi đầu vào.
- **Ứng dụng thực tiễn:** Kiến trúc này được sử dụng cho Dịch máy (Machine translation) và Nhận dạng giọng nói (Speech recognition).

Nền tảng Mạng Nơ-ron & Cách tiếp cận

- **Các loại mạng nơ-ron cơ sở:** Ba kiến trúc trên có thể được xây dựng từ nhiều loại mạng nơ-ron khác nhau, bao gồm:
 - **Transformer:** Kiến trúc phổ biến nhất hiện nay.
 - **LSTM:** Một dạng mạng nơ-ron hồi quy.
 - **State Space Models:** Một số kiến trúc mới gần đây.
- **Cách tiếp cận hộp đen:** Có thể coi bộ giải mã LLM như một "hộp đen" nhận đầu vào là chuỗi token và cho đầu ra là phân bố xác suất của các token để lấy mẫu.

2. Conditional Generation of Text: The Intuition

Ý tưởng cốt lõi

Một trực giác nền tảng của các mô hình ngôn ngữ là hầu hết mọi tác vụ ngôn ngữ đều có thể biểu diễn dưới dạng **sinh văn bản có điều kiện**.

- Mô hình sinh văn bản dựa trên một **prompt** (ngữ cảnh đầu vào)
- Văn bản được sinh ra từng token một
- Mỗi token phụ thuộc vào:
 - Prompt ban đầu
 - Các token đã sinh trước đó

Quy trình sinh có điều kiện

Quá trình sinh diễn ra bằng cách tính xác suất của token tiếp theo w_i từ ngữ cảnh trước đó

Công thức

$$P(w_i \mid w_{<i})$$

Cách hoạt động

- 1 Cung cấp prompt (ngữ cảnh đầu vào)
- 2 Mô hình tính xác suất token tiếp theo từ ngữ cảnh trước đó
- 3 Lấy mẫu từ phân phối trên để sinh token mới
- 4 Thêm vào ngữ cảnh
- 5 Lặp lại cho đến khi kết thúc (hoặc đạt độ dài mong muốn)

**Quá trình này lặp đi lặp lại tạo thành cơ chế tự hồi quy (autoregressive) của các mô hình ngôn ngữ lớn.*

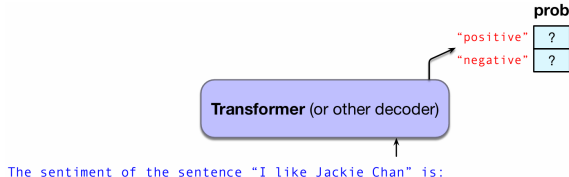
Ví dụ: Phân tích cảm xúc

Biến bài toán phân loại thành sinh văn bản

Thay vì huấn luyện classifier riêng, ta coi đây là bài toán sinh văn bản bằng cách đưa ra ngữ cảnh::

Prompt

The sentiment of the sentence "I like Jackie Chan" is:



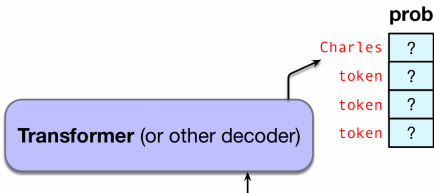
Quyết định

Mô hình so sánh $P(\text{positive} \mid \text{context})$ và $P(\text{negative} \mid \text{context})$ để chọn token cao hơn.

Ví dụ: Trả lời câu hỏi

Prompt

Q: Who wrote the book “The Origin of Species”? A:



Q: Who wrote the book 'The Origin of Species' A:

Quá trình sinh

- Bước 1: Tính xác suất $P(w|\text{context})$ để xem token nào có khả năng cao $\rightarrow$ chọn “Charles”
- Bước 2: Thêm “Charles” vào ngữ cảnh
- Bước 3: Tiếp tục dự đoán $\rightarrow$ chọn “Darwin”

Tại sao cách này hoạt động?

Mục tiêu huấn luyện

Mô hình được huấn luyện để:

- Dự đoán token tiếp theo trên tập dữ liệu rất lớn
- Học được:
 - Cấu trúc ngôn ngữ
 - Kiến thức thế giới (Ontological facts, lịch sử, toán học...)
 - Mẫu của các bài toán

Điểm quan trọng

Mô hình KHÔNG được huấn luyện riêng cho từng bài toán. Quá trình *pretraining* giúp nó có đủ kiến thức nền tảng.

3. Tinh chỉnh theo chỉ thị & Kỹ thuật Prompt

- **Instruction-tuning:** Quá trình huấn luyện bổ sung một mô hình ngôn ngữ cơ sở trên một tập dữ liệu đặc biệt gồm các chỉ thị (instructions) và câu trả lời tương ứng.
- **Prompt:** Là một chuỗi văn bản người dùng đưa cho LLM để yêu cầu mô hình thực hiện một tác vụ hữu ích.
- **Prompt engineering:** Quá trình tìm kiếm các prompt hiệu quả cho một tác vụ.
- **Ví dụ về các prompt đơn giản:**
 - Dạng câu hỏi: "What is a transformer network?" hoặc "Q: What is a transformer network? A:"
 - Dạng chỉ thị: "Translate the following sentence into Hindi: 'Chop the garlic finely'"

Prompt tường minh (Explicit Prompts)

- Các prompt tường minh xác định sẵn tập hợp các câu trả lời có thể xảy ra sẽ mang lại hiệu suất tốt hơn.
- **Ví dụ: Prompt template cho phân tích cảm xúc:**

Một bài đánh giá kèm theo một câu chưa hoàn chỉnh

Human: Do you think that “input” has negative or positive sentiment?

Choices:

(P) Positive

(N) Negative

Assistant: I believe the best answer is: (

- **Đặc điểm:** Xác định sẵn 2 lựa chọn (P) và (N), kết thúc bằng dấu ngoặc đơn mở (để ép mô hình trả lời (P) hoặc (N), và xác định rõ vai trò là Assistant.

Minh họa (Demonstrations) & Prompt Few-shot

Việc đưa thêm các ví dụ đã được gán nhãn vào trong prompt có thể cải thiện hiệu suất của mô hình:

- **Few-shot prompting:** Kỹ thuật prompt sử dụng các ví dụ minh họa (demonstrations).

Trái ngược với **Few-shot prompting:**

- **Zero-shot prompting:** Kỹ thuật đưa ra chỉ thị mà không kèm theo bất kỳ ví dụ minh họa nào.

Ví dụ: Prompt 2-shot

Các câu hỏi trắc nghiệm Khoa học máy tính THPT

The following are multiple choice questions about high school computer science.

Let $x = 1$. What is $x \ll 3$ in Python 3?

(A) 1 (B) 3 (C) 8 (D) 16

Answer: C

Which is the largest asymptotically?

(A) $O(1)$ (B) $O(n)$ (C) $O(n^2)$ (D) $O(\log(n))$

Answer: C

What is the output of the statement `'a' + 'ab'` in Python 3?

(A) Error (B) aab (C) ab (D) a ab

Answer: (Câu trả lời đúng mô hình cần sinh là (B))

Học trong ngữ cảnh (In-context Learning)

- **Đặc điểm của Demonstrations:**

- Có thể chọn thủ công hoặc tối ưu tự động (vd: dùng DSPy).
- Số lượng không cần lớn (quá nhiều gây ra "overfit" với ví dụ).
- Lợi ích chính là **minh họa định dạng đầu ra** hơn là đưa ra câu trả lời đúng.

- **In-context learning (Học trong ngữ cảnh):**

- Là quá trình học diễn ra trong lúc prompting, giúp cải thiện hiệu suất hoặc giảm loss, nhưng **không liên quan** đến việc cập nhật gradient cho các trọng số (parameters) của mô hình. (Trọng số không đổi, chỉ có ngữ cảnh và activation thay đổi).

Prompt Hệ thống (System Prompt) - Cơ bản

- **System prompt:** Một chuỗi văn bản là chỉ thị đầu tiên đưa cho LM, định nghĩa tác vụ, vai trò, giọng điệu và ngữ cảnh tổng thể. Được chèn âm thầm vào trước văn bản của người dùng.
- **Ví dụ về System prompt đơn giản:**

```
<system>You are a helpful and knowledgeable assistant.  
Answer concisely and correctly.
```

- **Khi kết hợp với truy vấn của người dùng:**

```
<system> You are a helpful and knowledgeable assistant.  
Answer concisely and correctly. <user> What is the capital  
of France?
```

Prompt Hệ thống dài & Phức tạp

- Các LLM hiện đại có ngữ cảnh rất lớn (hàng chục ngàn token), cho phép System prompt rất dài.
- **Ví dụ trích đoạn System prompt của Claude Opus 4 (dài 1700 từ):**
 - Claude should give concise responses to very simple questions, but provide thorough responses to complex and open-ended questions.
 - Claude is able to explain difficult concepts... with examples, thought experiments, or metaphors.
 - Claude does not provide information that could be used to make chemical or biological or nuclear weapons
 - ... keeps its tone natural, warm, and empathetic
 - ... avoids encouraging or facilitating self-destructive behavior
 - If Claude provides bullet points..., it should use markdown, and each bullet point should be at least 1-2 sentences long...

Prompt Hệ thống cho các Tác vụ cụ thể

- Có thể tạo System prompt cho các tác vụ chuyên biệt, ví dụ như kiểm tra ngữ pháp (grammar-checker).
- **Ví dụ System prompt kiểm tra ngữ pháp (Anthropic, 2025):**

Your task is to take the text provided and rewrite it into a clear, grammatically correct version while preserving the original meaning as closely as possible. Correct any spelling mistakes, punctuation errors, verb tense issues, word choice problems, and other grammatical mistakes.

- **Tổng kết:** Trong mọi trường hợp, prompt hệ thống luôn được nối vào trước các prompt hoặc truy vấn của người dùng, và toàn bộ chuỗi này đóng vai trò là ngữ cảnh (context) cho việc sinh văn bản có điều kiện của LLM.

4. Sinh văn bản và Lấy mẫu (Generation and Sampling)

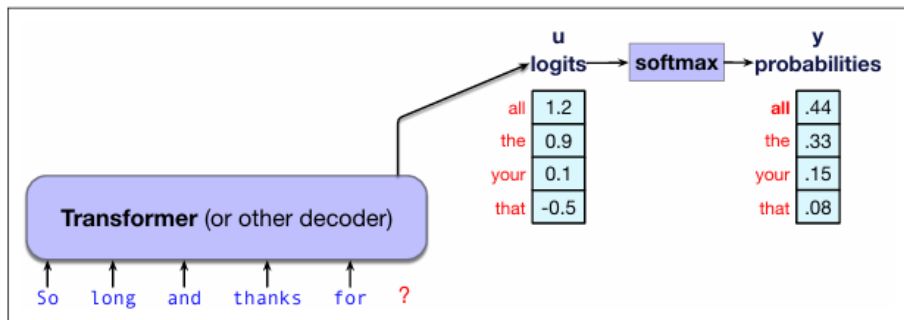
- **Câu hỏi đặt ra:** Tại mỗi bước, mô hình ngôn ngữ (Language Model - LM) nên sinh ra token nào?
- Quá trình sinh văn bản phụ thuộc vào xác suất của từng token.
- Các mạng nơ-ron bên trong mô hình ngôn ngữ (Transformers, LSTMs, State Space Models...) sẽ tạo ra các điểm số gọi là **logits** (là các số thực) cho mỗi token trong từ vựng.
- Vector điểm số u này sau đó được chuẩn hóa bằng hàm **softmax** để trở thành một phân bố xác suất hợp lệ.

Biến đổi Logits thành Xác suất

- Giả sử ta có một vector logit u có kích thước $[1 \times |V|]$ (cung cấp điểm số cho mỗi token tiếp theo có thể xảy ra).
- Ta truyền vector này qua hàm softmax để nhận được vector y , cũng có kích thước $[1 \times |V|]$, gán một xác suất cho từng token trong từ vựng.

$$y = \text{softmax}(u) \quad (7.1)$$

Ví dụ: Quá trình tính toán Softmax



- **Ngữ cảnh (Context):** "So long and thanks for ?"
- **Từ vựng đơn giản (4 từ):** *all, the, your, that.*
- **Vector Logits (u):** { all: 1.2, the: 0.9, your: 0.1, that: -0.5 }
- **Vector Xác suất (y) qua Softmax:** { all: .44, the: .33, your: .15, that: .08 }

Giải mã (Decoding) và Sinh Tự hồi quy

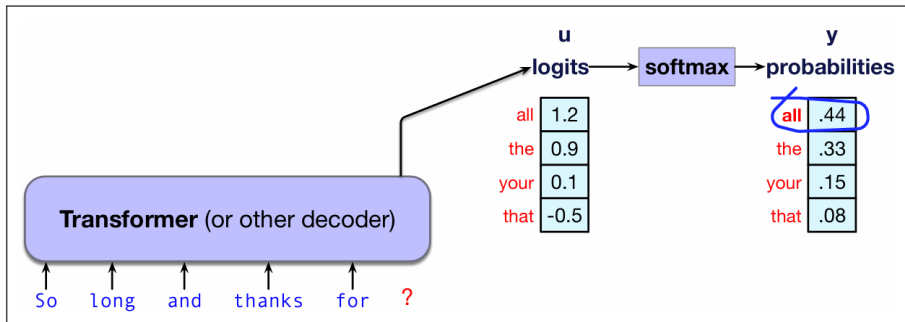
- **Giải mã (Decoding):** Là tác vụ chọn ra một token để sinh ra dựa trên các xác suất của mô hình.
- **Sinh tự hồi quy (Causal / Autoregressive generation):**
 - Quá trình giải mã từ một mô hình ngôn ngữ theo chiều từ trái sang phải (hoặc phải sang trái đối với tiếng Ả Rập).
 - Việc chọn token tiếp theo liên tục được *điều kiện hóa (conditioned)* dựa trên các lựa chọn trước đó của chúng ta.

Giải mã tham lam (Greedy decoding)

- **Khái niệm:** Cách đơn giản nhất để sinh token là **luôn luôn** sinh ra token có khả năng xảy ra cao nhất dựa vào ngữ cảnh.
- Đây là một thuật toán đưa ra lựa chọn tối ưu cục bộ tại mỗi bước.
- Tại mỗi bước thời gian t , ta biến các logits thành phân bố xác suất và chọn token đầu ra $\hat{w}_t$ trong từ vựng có xác suất cao nhất (argmax).

$$\hat{w}_t = \operatorname{argmax}_{w \in V} P(w | w_{<t}) \quad (7.2)$$

Ví dụ Giải mã tham lam



- Phân bố xác suất y :
 - all: .44
 - the: .33
 - your: .15
 - that: .08
- **Quyết định:** Thuật toán Greedy Decoding sẽ chọn token **all** vì nó có xác suất cao nhất (0.44).

Hạn chế của Giải mã tham lam

- **Không dùng trong thực tế cho LLM:** Phương pháp này thường không được sử dụng để sinh văn bản từ các mô hình ngôn ngữ lớn.
- **Nhược điểm:** Văn bản sinh ra có tính chung chung (generic) và thường lặp đi lặp lại.
- **Tính tất định (Deterministic):** Nếu ngữ cảnh giống hệt nhau, mô hình sẽ luôn sinh ra chuỗi văn bản hoàn toàn giống nhau.

Lấy mẫu ngẫu nhiên (Random sampling)

- **Khái niệm:** Chọn token tiếp theo để sinh ra dựa trên phân bố xác suất được mô hình gán cho nó.
- Thay vì luôn lấy từ cao nhất, ta lấy ngẫu nhiên tương tự như việc đổ xúc xắc (với xác suất thiên vị).
- Token có xác suất cao sẽ dễ được chọn hơn, nhưng token xác suất thấp vẫn có cơ hội được chọn.
- Còn được gọi là **random multinomial sampling** (Lấy mẫu đa thức ngẫu nhiên).

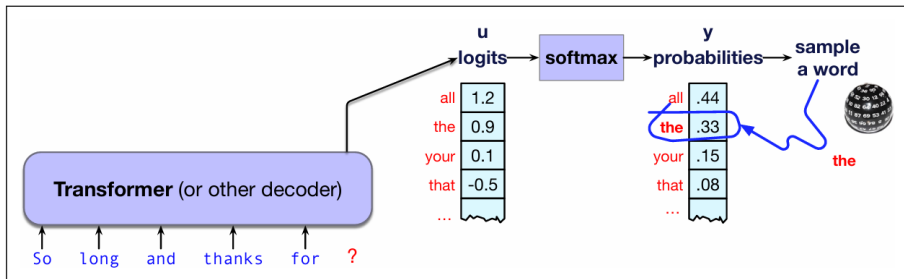
Thuật toán Lấy mẫu ngẫu nhiên

- Chúng ta sinh ra một chuỗi token $\{w_1, w_2, \dots, w_N\}$ cho đến khi gặp token kết thúc chuỗi (EOS - End Of Sequence).
- Ký hiệu $x \sim p(x)$ nghĩa là 'chọn x bằng cách lấy mẫu từ phân bố $p(x)$ '.

Thuật toán

```
 $i \leftarrow 1$   
 $w_i \sim p(w)$   
while  $w_i \neq \text{EOS}$   
     $i \leftarrow i + 1$   
     $w_i \sim p(w_i | w_{<i})$ 
```

Ví dụ & Hạn chế của Lấy mẫu ngẫu nhiên



- **Ví dụ:** Với phân bố $y = \{\text{all: .44, the: .33, ...}\}$, thuật toán có thể ngẫu nhiên chọn token **the** dù nó không phải cao nhất.
- **Vấn đề xảy ra:** Phân bố xác suất có phần "đuôi" chứa rất nhiều token kỳ lạ có xác suất thấp. Dù mỗi token hiếm có xác suất nhỏ, nhưng tổng xác suất của tất cả chúng cộng lại là đủ lớn khiến chúng thỉnh thoảng được chọn, dẫn đến các câu kỳ quặc.
- **Kết luận:** Greedy thì quá nhàm chán, còn Random thì quá ngẫu nhiên.

- **Ý tưởng:** Định hình lại (reshape) phân bố xác suất để:
 - *Tăng* xác suất của các token có khả năng cao.
 - *Giảm* xác suất của các token có khả năng thấp.
- **Kết quả:** Ít có khả năng sinh ra các token hiếm (kỳ quặc) và tăng khả năng sinh ra các token tốt.

Công thức Temperature sampling

- Ta chia logit cho một tham số Temperature τ trước khi truyền qua hàm softmax.

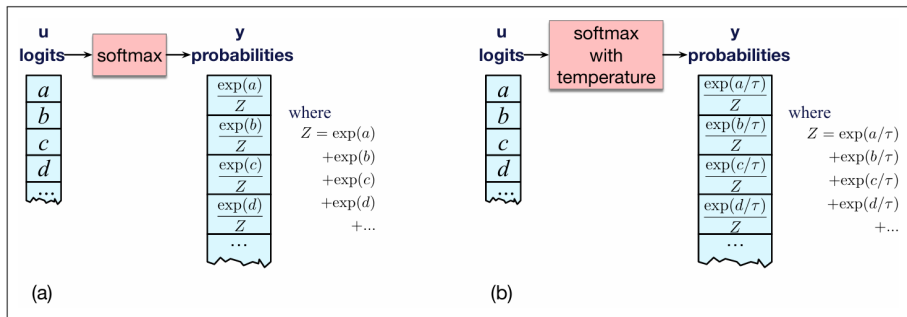
Softmax thông thường

$$y = \text{softmax}(u) \quad (7.3)$$

Softmax với Temperature τ

$$y = \text{softmax}(u/\tau) \quad (7.4)$$

Phân tích Toán học của Temperature



- Việc chia cho số $\tau < 1$ làm **tăng độ lớn** của các điểm số truyền vào softmax.
- Đặc tính Softmax: số càng lớn đẩy về 1, số nhỏ đẩy về 0.
- **Do đó**, khi truyền các số lớn hơn vào hàm softmax, kết quả là một phân phối với xác suất tăng lên của các từ có xác suất cao nhất và xác suất giảm xuống của các từ có xác suất thấp, phân bố trở nên "tham lam" (greedy) hơn.

Ví dụ & Hệ quả việc thay đổi giá trị τ

softmax output with temperature τ

	logits	close to greedy		normal softmax		close to uniform
		$\tau=0.1$	$\tau=0.5$	$\tau=1$	$\tau=10$	$\tau=100$
all	1.2	.95	.59	.44	.27	.25
the	0.9	.05	.32	.33	.26	.25
your	0.1	0	.07	.15	.24	.25
that	-0.5	0	.02	.08	.23	.25

- Khi $\tau = 1$: Là hàm softmax thông thường. Phân bố không thay đổi.
- Khi τ tiến dần về 0: Việc chia cho τ làm xác suất của từ xuất hiện nhiều nhất tiến dần đến 1 $\Rightarrow$ Kết quả tiệm cận với **Greedy decoding**.
- **Lấy mẫu Temperature cao**: Khi $\tau > 1$, việc chia cho τ sẽ làm *phẳng* phân bố xác suất, tăng tính ngẫu nhiên.

Quizz Kahoot